COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	09/04/2026	**DATE SUBMITTED:**	09/08/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 7.1 **Sorting Algorithms**

1. PROCEDURE OUTPUT

Code Snippet

```
6    const int max_size = 100;
7
8    void displayAll(int arr[]);
```

```
8   void displayAll(int arr[]);
9
10  int main() {
11
12      int dataset[max_size];
13      srand(time(0));
14
15      for (int i = 0; i < max_size; i++){
16          dataset[i] = rand()%100;
17      }
18      displayAll(dataset);
```

```
49  void displayAll(int arr[]) {
50      for (int i = 0; i < max_size; i++) {
51      std::cout << arr[i] << ", ";
52      }
```

Output Console Screenshot

```
37, 7, 18, 71, 14, 33, 57, 87, 92, 76, 43, 75, 94, 42, 38, 18, 76, 48, 5, 28, 48, 2, 32, 89, 95, 74, 50, 80, 4
5, 0, 3, 16, 68, 4, 96, 46, 26, 32, 15, 88, 49, 0, 3, 50, 66, 91, 44, 40, 25, 48, 46, 35, 56, 5, 96, 79, 18, 5
4, 7, 87, 95, 97, 56, 83, 85, 56, 63, 87, 88, 44, 24, 44, 41, 41, 87, 21, 67, 19, 78, 70, 84, 11, 14, 47, 87,
16, 19, 47, 77, 31, 4, 47, 61, 57, 55, 26, 35, 88, 72, 5,
```

Observations

In the first procedural table activity, we first created an array of elements with random values and the array must contain a max element of 100 values. It is stated that it should not be sorted, Therefore, the algorithm for the creation for the printing of array of values is,

#A max size array of 100
#A for loop that will generate 100 random numbers
#printing loop should be integrated inside a function
#print the 100 data in the main.cpp file by function.

Which is now represented as the code implementation above. First is the constant declaration for the **max_size array of 100 random values**. Next is the declaration of the **function prototype for the needed parameter** of the function, and on the bottom part of the main.cpp file, is the entire function definition for the processing of the printing of the function **displayAll()**. On the main block of code. Is the generating of 100 types of random values using the **rand()** integrated with **srand(time)** for the processing of generating random values. Once data is generated, the displayAll function is called for the printing of the 100 random values.

Table 7-1. Array of Values for Sort Algorithm Testing

Code Snippet

```
4   template <typename T>
5   void bubbleSort (T arr[], int maxSize){
6
7       for(int i = 0; i < maxSize; i++) {
8           for(int j = i + 1; j < maxSize; j++){
9               if(arr[i] > arr[j]){
10                  std::swap(arr[j], arr[i]);
11              }
12          }
13      }
14  }
```

Output Console Screenshot

```
Bubble sort Testing:
0, 0, 2, 3, 3, 4, 4, 5, 5, 5, 7, 7, 11, 14, 14, 15, 16, 16, 18, 18, 18, 19, 19, 21, 24, 25, 26, 26, 28, 31, 32
, 32, 33, 35, 35, 37, 38, 40, 41, 41, 42, 43, 44, 44, 44, 45, 46, 46, 47, 47, 47, 48, 48, 48, 49, 50, 50, 54,
55, 56, 56, 56, 57, 57, 61, 63, 66, 67, 68, 70, 71, 72, 74, 75, 76, 76, 77, 78, 79, 80, 83, 84, 85, 87, 87, 87
, 87, 87, 88, 88, 88, 89, 91, 92, 94, 95, 95, 96, 96, 97,
```

Observations

In the Bubble Sort algorithm, the way it sorts the data is that it compares 2 values together. If the first array value is greater than to the second array value, the condition will run the swapping method making the first array value equivalent to the former second array value. The swapped value will become its new basis to compare another value as j continues to iterate on the entire list of the array.

Table 7-2. Bubble Sort Technique

Code Snippet

```
18  template <typename T>
19  int routineSmallest(T arr[], int K, int maxSize){
20      int position;
21      T smallestElem = arr[K];
22      position = K;
23
24      for (int j = K + 1; j < maxSize; j++){
25          if(arr[j] < smallestElem){
26              smallestElem = arr[j];
27              position = j;
28          }
29      }
30      return position;
31  }
32
33
34
35  template <typename T>
36  void selectionSort(T arr[], int maxSize){
37      int POS;
38      for (int i = 0; i < maxSize; i++) {
39          POS = routineSmallest(arr, i, maxSize);
40          std::swap(arr[i], arr[POS]);
41      }
42  }
```

Output Console Screenshot

```
 Selection Sort testing:
0, 0, 2, 3, 3, 4, 4, 5, 5, 5, 7, 7, 11, 14, 14, 15, 16, 16, 18, 18, 18, 19, 19, 21, 24, 25, 26, 26, 28, 31, 32
, 32, 33, 35, 35, 37, 38, 40, 41, 41, 42, 43, 44, 44, 44, 45, 46, 46, 47, 47, 47, 48, 48, 48, 49, 50, 50, 54,
55, 56, 56, 56, 57, 57, 61, 63, 66, 67, 68, 70, 71, 72, 74, 75, 76, 76, 77, 78, 79, 80, 83, 84, 85, 87, 87, 87
, 87, 87, 88, 88, 88, 89, 91, 92, 94, 95, 95, 96, 96, 97,
```

Observations

The selection sort algorithm uses a helper function to keep track of the smallest value to switch it up to the very beginning of the list as the algorithm compares elements from the unsorted portion of the list. When the smallest the algorithm already has the smallest value or the minimum value of the list, it swaps it to the very beginning of the first loop iteration and identifies it as sorted. Therefore, lessening the need to iterate by 1 makes it easier to loop when the list gets smaller and smaller.

Table 7-3 Selection Sort Algorithm

Code Snippet

```
46  template <typename T>
47  void insertionSort(T arr[], int maxSize){
48      int k=1, j, temp;
49
50      while (k < maxSize){
51          temp = arr[k];
52          j = k - 1;
53
54          while(temp <= arr[j]){
55              arr[j+1] = arr[j];
56              j--;
57          }
58          arr[j+1] = temp;
59          k++;
60      }
61  }
```

Output Console Screenshot

```
 Insertion Sort testing:
0, 0, 2, 3, 3, 4, 4, 5, 5, 5, 7, 7, 11, 14, 14, 15, 16, 16, 18, 18, 18, 19, 19, 21, 24, 25, 26, 26, 28, 31, 32
, 32, 33, 35, 35, 37, 38, 40, 41, 41, 42, 43, 44, 44, 44, 45, 46, 46, 47, 47, 47, 48, 48, 48, 49, 50, 50, 54,
55, 56, 56, 56, 57, 57, 61, 63, 66, 67, 68, 70, 71, 72, 74, 75, 76, 76, 77, 78, 79, 80, 83, 84, 85, 87, 87, 87
, 87, 87, 88, 88, 88, 89, 91, 92, 94, 95, 95, 96, 96, 97,
```

Observations
The insertion sort algorithm as discussed, it was stated that it is just how people group their cards in a card game. The first card on the very left of your handful of cards, you treat it as a sorted value first before proceeding to the other cards that you have. Once you see a value that is close to the value, you either put it greater between the cards or less than to the first card you have treated as sorted. For example, 6, 8, 10, 5, 7, 11 The 6, 8, 10 is pretty much sorted as comparing that 10 is greater than 8, and 8 is greater than 6, but when it comes to the 4th iteration, 6, 8, 10 is greater than 5, therefore, we will insert 5 before 6 since it's less than 6. For 7, 6 is less than 7, but 7 is less than 8, therefore we will insert the 7 between 6 and 8. That's how insertion works. It just keeps iterating and sorting the value where it compares the unsorted value to the sorted part of the list.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE OF MAIN CPP

```
DEFINE CONSTANT max_size = 100

DEFINE ARRAY candidateNames = [
   "Bo Dalton Capistrano",
   "Cornelius Raymon Agustín",
   "Deja Jayla Bañaga",
   "Lalla Brielle Yabut",
   "Franklin Relano Castro"
]

FUNCTION main():
   DECLARE ARRAY votes of size max_size
   SEED random number generator with current time

   FOR i FROM 0 TO max_size - 1 DO                    // generate array with random votes from 1 to 5
      votes[i] = random integer in range [1, 5]
   END FOR

   PRINT "Unsorted Votes:"
   CALL displayAll(votes)

   // Sort, tally, and get winning candidate index
   SET winningIndex = CALL majorityVote(votes, max_size)

   PRINT "Winning Candidate: " + candidateNames[winningIndex]
END FUNCTION


FUNCTION displayAll(arr):
   FOR i FROM 0 TO max_size - 1 DO
      PRINT arr[i] + " "
   END FOR
END FUNCTION


FUNCTION majorityVote(voteArray, arrSize):
   DECLARE ARRAY voteCount of size 5 initialized to [0, 0, 0, 0, 0]

   CALL insertionSort(voteArray, arrSize)          // Sort votes in ascending order using insertion sort header

   FOR i FROM 0 TO arrSize - 1 DO                  // Count vote frequency for each candidate
      SET candidateID = voteArray[i]
      INCREMENT voteCount[candidateID - 1] BY 1
```

```
    END FOR

    SET max = voteCount[0]
    SET winner = 0

    PRINT "--- | Vote Tallying Breakdown | ---"

    FOR i FROM 0 TO 4 DO                    // Print breakdown and determine maximum votes
        IF voteCount[i] > max THEN
            SET max = voteCount[i]
            SET winner = i
        END IF

        PRINT candidateNames[i] + ": " + voteCount[i] + " votes"
    END FOR

    RETURN winner
END FUNCTION
```

PSEUDOCODE OF HEADER FILE

```
FUNCTION insertionSort(arr, maxSize):
    SET k = 1

    WHILE k < maxSize DO
        SET temp = arr[k]                   // Store the current element to insert
        SET j = k - 1                       // Index of the element right before k

        WHILE j > -1 AND temp <= arr[j] DO  // Shift elements of arr[0..k-1] that are greater than or equal to temp
            arr[j + 1] = arr[j]             // one position ahead
            DECREMENT j by 1
        END WHILE

        arr[j + 1] = temp                   // Place temp into its correct sorted position
        INCREMENT k by 1
    END WHILE
END FUNCTION
```

CODE OF THE MAIN.CPP ALGORITHM

```cpp
#include <iostream>
#include "votesSort.h"
#include <cstdlib>
#include <ctime>
#include <string>

const int max_size = 100;

// Array containing the list of candidates to iterate for the winner
const std::string candidateNames[5] = {
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustín",
    "Deja Jayla Bañaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

void displayAll(int arr[]);
int majorityVote(int voteArray[], const int& arrSize);

int main(){

    int votes[max_size];
    srand(time(0));

    for (int i = 0; i < max_size; i++){
        votes[i] = rand()%5 + 1;
    }

    // Displays the unsorted votes first
    std::cout << "Unsorted Votes:\n";
    displayAll(votes);
    std::cout << "\n\n";

    // Calls function to sort, count, and find the winner
    int winningIndex = majorityVote(votes, max_size);
```

```cpp
    // Print the winner's name using the returned index
    std::cout << "\n🏆 Winning Candidate: " << candidateNames[winningIndex]
    << " (ID: " << winningIndex + 1 << ")\n";

    return 0;
}

void displayAll(int arr[]){
    for (int i = 0; i < max_size; i++){
        std::cout << arr[i] << " ";
    }
}

int majorityVote(int voteArray[], const int& arrSize){
    int voteCount[5] = {0};
    insertionSort(voteArray, arrSize);

    // Count occurrences
    for (int i = 0; i < arrSize; i++){
        voteCount[voteArray[i] - 1]++;
    }

    // Find max
    int max = voteCount[0];
    int winner = 0;

    std::cout << "--- | Vote Tallying Breakdown | ---\n\n";
    for (int i = 0; i < 5; i++){
        if (voteCount[i] > max){
            max = voteCount[i];
            winner = i;
        }
        // Print the counts with the names alongside them
        std::cout << candidateNames[i] << " (" << i+1 << "): " << voteCount[i] << " votes\n";
    }
    return winner;
}
```

CODE OF THE HEADER FILE ALGORITHM

```cpp
#ifndef VOTESSORT_H
#define VOTESSORT_H

template <typename T>
void insertionSort(T arr[], int maxSize){
    int k = 1, j, temp;

    while (k < maxSize){
        temp = arr[k];
        j = k - 1;

        while(temp <= arr[j] && j > -1){
            arr[ j + 1 ] = arr[j];
            j--;
        }
        arr[ j + 1 ] = temp;
        k++;
    }
}

#endif
```

OUTPUT

```
Unsorted Votes:
1 2 2 3 5 5 3 4 3 3 4 3 2 5 5 1 4 4 4 5 5 2 3 3 4 5 5 4 2 1 5 5 1 2 5 4 2 3 2 2 2 1 4 1 5 3 3
 1 4 3 2 3 2 1 2 3 5 3 3 3 5 2 1 4 2 4 1 1 3 1 3 4 2 3 1 2 2 3 3 3 1 5 5 5 3 4 5 3 4 3 2 5 5
4 2 4 1 1 2 1

Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3
 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5 5

--- | Vote Tallying Breakdown | ---

Bo Dalton Capistrano (1): 17 votes
Cornelius Raymon Agustín (2): 21 votes
Deja Jayla Bañaga (3): 25 votes
Lalla Brielle Yabut (4): 17 votes
Franklin Relano Castro (5): 20 votes

🏆 Winning Candidate: Deja Jayla Bañaga (ID: 3)
```

CONCLUSION AND ANALYSIS

In this supplementary activity, I used insertion sort in sorting the votes since it has the less time complexity as per calculated compared to the other sorting algorithm techniques. In order to accomplish this supplementary activity, we have have **3 arrays in total**, **one for the 100 randomly generated values between 1-5** identified as a vote, **one for the names of the candidates**, to be able **to display their name beside their votes**, And an **array for the storing of votes**, after sorting the randomly generated array votes and counting the votes per candidates. **A total of 3 functions**, **displayAll function for the generated votes**, and **majorityVote for the counting, sorting, and printing of the winner and votes for each candidate**, and **the sorting algorithm function insertionSort()** inside the majorityVote function. In the header file is the very sorting algorithm before the counting begins for each candidate. The function counts the total occurrences of each number as per iteration, and it identifies which is the highest vote count. Therefore, the candidate with the highest vote count, will be the returned value as the winner variable.

Overall, this activity has further polished my understanding on applying for loops and integrating it with sorting algorithms, to further provide a detailed explanation and representation of how to organize data without hassles. This activity was fun given that I also had to use some of the codes from the previous HOA activities, which entirely integrates all my learnings into one and unravels new ideas.

ID	Manual Count	Count Result of the Algorithm
1	17 votes	17 votes
2	21 votes	21 votes
3	25 votes	25 votes
4	17 votes	17 votes
5	20 votes	20 votes

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.